Fresh: probador virtual de barbería con computación gráfica

Proyecto final de Computación Gráfica (CS4016)

Kalos Lazo Mera Sofía Herrera Gianpier Segovia

Universidad de Ingeniería y Tecnología (UTEC)

Noviembre de 2026

Resumen

Fresh (repositorio *BarberStudio*) es una aplicación web que permite probarse barbas, tintes y cortes de pelo sobre una foto propia o con la cámara, antes de ir a la barbería. A partir de los 478 puntos faciales que entrega un modelo preentrenado, el sistema construye una malla de triángulos con una triangulación de Delaunay implementada desde cero (algoritmo de Fortune), deforma texturas de barba real por triángulo con interpolación bilineal propia, recolorea el pelo en el espacio CIELAB, quita y vuelve a crear el pelo para ocho cortes con un relleno *pull-push* y un modelo de cráneo ajustado a cada cabeza, y representa caras, barbas y pelucas como miles de gaussianas en 2D y en 3D, con un visor de *Gaussian Splatting* escrito en WebGL2. El proyecto incluye simulación física de la barba con integración de Verlet, un juego de afeitado, una clasificación de la forma de la cara que sugiere estilos, la segmentación del pelo con un YOLOv8 ajustado por el equipo (mAP50 de máscara de 0,968) y una ficha con indicaciones para el barbero. Se valida con 47 pruebas automáticas, integración continua y un contenedor Docker que reproduce la aplicación con un solo comando.

Palabras clave: triangulación de Delaunay, warp afín por partes, espacio de color CIELAB, composición alfa, *pull-push*, *Gaussian Splatting*, integración de Verlet, segmentación de pelo.

Índice

1. Introducción **4**

1.1. Problema 4

1.2. Objetivos 4

1.3. Contribuciones 4

1.4. Organización del documento 5

2. Fundamentos **5**

2.1. Imágenes, espacios de color y composición 5

2.2. Muestreo, interpolación y coordenadas baricéntricas 6

2.3. Triangulación de Delaunay y algoritmo de Fortune 6

2.4. Transformación afín por triángulo 6

2.5. Gaussianas, *splatting* y proyección EWA 6

2.6. Simulación con Verlet y dinámica basada en posiciones 7

2.7. Rellenado de regiones . . . 7
2.8. Orientación local: el tensor de estructura . . . 7
2.9. Detección y segmentación con modelos . . . 7

3. Requisitos y metodología **8**
3.1. Requisitos . . . 8
3.2. Metodología . . . 8
3.3. Organización del equipo . . . 9

4. Arquitectura del sistema **9**
4.1. Servidor . . . 9
4.2. Interfaz . . . 10
4.3. Flujo de una petición . . . 10

5. Implementación **11**
5.1. Detección de la cara y de los puntos faciales . . . 11
5.2. Segmentación del pelo . . . 12
5.3. Forma de la cara y sugerencias . . . 12
5.4. Barba con plantillas reales . . . 13
5.5. Tinte en CIELAB . . . 14
5.6. Motor de cortes de pelo . . . 15
5.7. Tinte también en la barba . . . 17
5.8. Combinación de looks y ficha para el barbero . . . 18
5.9. Física de la barba . . . 18
5.10. Gaussianas 2D: "¿Cuántas gaussianas eres?" . . . 19
5.11. Espejo 3D con *Gaussian Splatting* . . . 20
5.12. Peluca 3D multivista . . . 21
5.13. Laboratorio: avatar en vivo . . . 22
5.14. Contrarreloj: el juego de afeitado . . . 22

6. Herramientas utilizadas **23**

7. Cómo reproducir el proyecto **23**
7.1. Opción A: con Docker (recomendada) . . . 24
7.2. Opción B: con Python . . . 24
7.3. Modelos . . . 25
7.4. Pruebas . . . 25
7.5. Reentrenar el segmentador de pelo . . . 25
7.6. Usar la aplicación . . . 25

8. Resultados **26**

8.1. Cortes de pelo 26
8.2. Tinte y segmentación del pelo 26
8.3. Tinte de la barba 27
8.4. Gaussianas 2D 27
8.5. Contrarreloj 28
8.6. Interfaz 28
8.7. Rendimiento 29
8.8. Pruebas automáticas 29

9. Limitaciones y trabajo futuro **30**

10. Conclusiones **31**

A. API del servidor **34**

B. Estructura del código **34**

1. Introducción

1.1. Problema

Cambiar de corte, dejarse la barba o teñirse el pelo son decisiones que se toman a ciegas: el resultado solo se ve cuando ya está hecho y, en el caso de un corte o un tinte, no se puede deshacer. Las aplicaciones que existen para probarse estilos suelen pegar una imagen recortada encima de la foto, sin seguir la forma de la cara ni la luz, o dependen de modelos generativos que inventan la cara completa y requieren servidores con GPU.

Este proyecto aborda el problema con técnicas de computación gráfica: en lugar de generar una imagen nueva, se modela la geometría de la cara y del pelo de la persona y se componen sobre la foto original las capas necesarias (barba, color, pelo corto, piel), de forma que el resultado conserve la identidad, la iluminación y la textura de la foto.

1.2. Objetivos

Objetivo general. Construir un producto completo, en forma de aplicación web, que permita probar barbas, tintes y cortes de pelo sobre una foto o con la cámara, usando como núcleo algoritmos de computación gráfica implementados por el equipo.

Objetivos específicos.

1. Ajustar plantillas de barba real a cualquier cara mediante una malla de Delaunay propia y un *warp* afín por triángulo.
2. Recolorear el pelo de forma creíble, respetando su textura y su iluminación.
3. Simular cortes de pelo cortos (degradados, *taper*, *undercut*, raya marcada, *crew cut* y rapado) quitando el pelo original y reconstruyendo el fondo y la piel.
4. Representar la cara y el pelo con gaussianas en 2D y en 3D, y renderizarlas en el navegador.
5. Añadir interacción en tiempo real: modo en vivo con la cámara, física de la barba y un juego de afeitado.
6. Entregar el sistema con pruebas, integración continua, documentación y un contenedor reproducible.

1.3. Contribuciones

Las piezas de visión que detectan la cara y sus puntos (MediaPipe) y la segmentación base del pelo se toman de modelos preentrenados; todo lo demás es implementación propia. En particular:

- triangulación de Delaunay con el algoritmo de barrido de Fortune, *warp* afín por triángulo, prueba de interior con coordenadas baricéntricas e interpolación bilineal (sección 5.4);
- tinte en CIELAB que separa la luminosidad base del detalle de cada mechón (sección 5.5);
- motor de cortes con relleno *pull-push*, modelo de cráneo, línea del pelo a partir de la frente y sombreado de Lambert (sección 5.6);
- detección de barba real por textura y tensor de estructura, y clasificación de la forma de la cara (secciones 5.7 y 5.3);
- gaussianas 2D ajustadas con gradientes analíticos y Adam escrito a mano, *Gaussian Splatting* 3D con proyección EWA en WebGL2, exportación a `.ply` y peluca 3D multivista (secciones 5.10 a 5.12);

- física de la barba con Verlet y restricciones de distancia, y el juego de afeitado con detección de corte por cápsula (secciones 5.9 y 5.14);
- ajuste fino de YOLOv8 para segmentar pelo con Figaro1k y CelebAMask-HQ (sección 5.2).



Figura 1: Pantalla de inicio de Fresh. La demostración de la derecha muestra un estilo de barba aplicado sobre un retrato de dominio público de la NASA.

1.4. Organización del documento

La sección 2 resume los fundamentos teóricos. La sección 3 describe los requisitos y la metodología de trabajo, y la sección 4, la arquitectura del sistema. La sección 5 detalla cada módulo con sus algoritmos, decisiones y limitaciones. La sección 6 indica qué herramienta resolvió cada parte, y la sección 7 explica paso a paso cómo reproducir el proyecto. Los resultados se presentan en la sección 8, las limitaciones en la sección 9 y las conclusiones en la sección 10.

2. Fundamentos

Esta sección resume la teoría que usa el proyecto, con la notación que se emplea en el resto del documento.

2.1. Imágenes, espacios de color y composición

Una imagen digital es una función muestreada $I : \{0, \dots, W-1\} \times \{0, \dots, H-1\} \to [0, 255]^3$ [1]. Además de RGB, el proyecto usa dos espacios de color:

- **YCrCb**, que separa la luminancia Y de dos componentes de croma C_r y C_b. Permite cambiar el tono de una barba sin alterar su brillo.
- **CIELAB**, diseñado para que distancias iguales correspondan aproximadamente a diferencias de color percibidas iguales [2, 3]. L^* es la luminosidad, y a^* y b^* son los ejes verde–rojo y

azul–amarillo. Cambiar a^*, b^* manteniendo la variación de L^* recolorea sin perder la textura.

La composición de una capa de color c con opacidad $\alpha \in [0,1]$ sobre un fondo b sigue el operador *over* de Porter y Duff [4]:

$$C = \alpha\, c + (1-\alpha)\, b. \tag{1}$$

Para la barba también se usa el modo *multiply*, $C = (1-\alpha)\, b + \alpha\, b\, m$ con $m \in [0,1]$, que oscurece la piel sin tapar sus luces y sombras.

2.2. Muestreo, interpolación y coordenadas baricéntricas

Cuando una transformación geométrica lleva un píxel de salida a una posición no entera (x, y) de la imagen de origen, el valor se reconstruye con interpolación bilineal [5]. Con $x_0 = \lfloor x \rfloor$, $y_0 = \lfloor y \rfloor$, $w_x = x - x_0$ y $w_y = y - y_0$:

$$v = (1-w_x)(1-w_y)P_{00} + w_x(1-w_y)P_{10} + (1-w_x)w_y P_{01} + w_x w_y P_{11}. \tag{2}$$

Un punto p dentro de un triángulo ABC se escribe como $p = \lambda_1 A + \lambda_2 B + \lambda_3 C$, con $\lambda_1 + \lambda_2 + \lambda_3 = 1$. Estas son sus coordenadas baricéntricas: el punto está dentro del triángulo si y solo si $\lambda_i \geq 0$ para todo i. Además, si los vértices se mueven, $p' = \lambda_1 A' + \lambda_2 B' + \lambda_3 C'$ es el mismo punto "pegado" a la superficie. El proyecto usa esta propiedad para anclar pelos y gaussianas a la cara.

2.3. Triangulación de Delaunay y algoritmo de Fortune

Dado un conjunto de puntos P en el plano, la triangulación de Delaunay es aquella en la que ningún punto de P cae dentro del círculo circunscrito de ningún triángulo [6]. Entre todas las triangulaciones posibles, maximiza el ángulo mínimo [7]; por eso evita triángulos muy finos, que deformarían mal una textura. Es el grafo dual del diagrama de Voronoi.

El algoritmo de Fortune [8] construye el diagrama de Voronoi en $O(n \log n)$ con una línea de barrido. Mantiene una *beach line* formada por arcos de parábola, uno por cada sitio ya visitado, y procesa dos tipos de eventos en una cola de prioridad:

- **evento de sitio**: la línea alcanza un punto nuevo y aparece un arco;
- **evento de círculo**: un arco se reduce a ancho cero y desaparece. Su posición es un vértice de Voronoi, y los tres sitios que lo definen tienen un círculo circunscrito vacío, es decir, forman un triángulo de Delaunay.

Las alternativas incrementales, como Bowyer–Watson [9, 10], insertan los puntos uno por uno y rehacen los triángulos cuyo círculo contiene el punto nuevo.

2.4. Transformación afín por triángulo

Una transformación afín $T(x, y) = (ax + by + c,\ dx + ey + f)$ queda determinada por tres pares de puntos. Por eso, una malla de triángulos en dos posiciones (origen y destino) define un *warp* afín por partes [11]: cada triángulo de la textura se lleva a su triángulo correspondiente sobre la cara. Para no dejar huecos, se recorre cada píxel del triángulo de destino y se calcula con la transformación inversa de dónde leer en el origen (*backward mapping*).

2.5. Gaussianas, *splatting* y proyección EWA

Una gaussiana anisótropa 3D con centro μ y covarianza Σ tiene densidad $G(x) = \exp\!\Big(-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\Big)$. *3D Gaussian Splatting* [12] representa una escena como un

conjunto de gaussianas con color y opacidad. Para dibujarlas, cada gaussiana se proyecta a la pantalla con la aproximación EWA [13]: si W es la rotación de la vista y J la jacobiana de la proyección en el centro, la covarianza en pantalla es

$$\Sigma_{2D} = J\,W\,\Sigma\,W^{\top}J^{\top}. \tag{3}$$

Las gaussianas proyectadas se ordenan de atrás hacia adelante y se componen con

$$C = \sum_i c_i\,\alpha_i \prod_{j<i}(1-\alpha_j). \tag{4}$$

En 2D, GaussianImage [14] representa una imagen como suma de gaussianas sin orden, ajustadas por descenso de gradiente. El proyecto implementa ambas ideas desde cero, sin bibliotecas de diferenciación automática.

2.6. Simulación con Verlet y dinámica basada en posiciones

La integración de Verlet [15] avanza una partícula usando su posición actual y la anterior: $x_{t+h} = x_t + (x_t - x_{t-h}) + a\,h^2$. No guarda la velocidad explícitamente, por lo que imponer restricciones consiste simplemente en mover posiciones. La dinámica basada en posiciones (PBD) [16, 17] aprovecha esto: después de integrar, proyecta las partículas para que cumplan restricciones como la distancia fija entre nodos de una cuerda. Es estable incluso con pasos de tiempo grandes, algo necesario a 10 cuadros por segundo.

2.7. Rellenado de regiones

Cuando se quita pelo que tapaba el fondo, hay que inventar los píxeles faltantes. El *inpainting* por *fast marching* de Telea [18] propaga los valores del borde hacia adentro. El método *pull-push* [19] construye una pirámide: en la fase *pull* promedia, en cada nivel, solo los píxeles conocidos ponderados por su peso; en la fase *push* rellena los huecos de cada nivel fino con el nivel grueso. La diferencia clave para este proyecto es que en *pull-push* se elige qué píxeles cuentan como conocidos.

2.8. Orientación local: el tensor de estructura

Con los gradientes g_x, g_y de una imagen, el tensor de estructura promedia en una vecindad la matriz $J = \begin{pmatrix} g_x^2 & g_x g_y \\ g_x g_y & g_y^2 \end{pmatrix}$ [20]. Sus autovalores $\lambda_1 \geq \lambda_2$ describen la textura local:

- la **orientación dominante** es $\theta = \frac{1}{2}\,\mathrm{atan2}(2J_{xy}, J_{xx} - J_{yy})$;
- la **coherencia** $(\lambda_1 - \lambda_2)/(\lambda_1 + \lambda_2)$ vale cerca de 1 en un borde recto y largo, y es menor en una textura de trazos en todas direcciones, como la barba.

2.9. Detección y segmentación con modelos

El detector de Viola–Jones [21] clasifica ventanas con una cascada de rasgos de Haar. MediaPipe [22] ofrece, entre otros, un modelo de malla facial de 468 puntos con profundidad, más 10 puntos de iris [23], y un segmentador por clases (fondo, pelo, piel del cuerpo, piel de la cara, ropa y accesorios). YOLO [24] es una familia de detectores de una sola pasada; YOLOv8 [25] agrega una cabeza de segmentación por instancia. Las máscaras gruesas se ajustan a los bordes reales con el filtro guiado de He *et al.* [26], que usa la propia foto como guía.

3. Requisitos y metodología

3.1. Requisitos

Funcionales.

RF1. Cargar una foto, tomarla con la cámara o usar una foto de ejemplo, y compartirla entre todas las herramientas.

RF2. Aplicar cuatro estilos de barba con densidad ajustable.

RF3. Teñir el pelo con siete colores o un color libre y, opcionalmente, también la barba.

RF4. Aplicar ocho cortes: degradado bajo, medio y alto, *taper*, *undercut*, raya marcada, *crew cut* y rapado.

RF5. Combinar barba, tinte y corte en un mismo resultado.

RF6. Comparar antes y después, guardar looks y generar una ficha para el barbero.

RF7. Sugerir estilos según la forma de la cara.

RF8. Mostrar la cabeza y una peluca en 3D, girables y exportables.

RF9. Probar estilos en vivo con la cámara, con física de la barba y un juego de afeitado.

No funcionales y restricciones.

RNF1. Sin GPU: todo el procesamiento corre en CPU.

RNF2. Respuesta de menos de 2 s por foto en el contenedor, para fotos de celular de hasta 4000 px.

RNF3. Interfaz sin paso de compilación (HTML, CSS y JavaScript nativos), para que el proyecto sea fácil de reproducir en el curso.

RNF4. La interfaz se adapta al celular.

RNF5. Reproducible con un solo comando mediante Docker.

3.2. Metodología

El equipo trabajó de forma iterativa e incremental, con entregas funcionales en cada iteración. Cada funcionalidad nueva siguió el mismo ciclo: propuesta escrita en `docs/`, implementación en `backend/app/core`, pruebas automáticas, revisión visual sobre retratos de prueba y documentación técnica. Las fechas salen del historial de `git`:

Iteración	Contenido
1 (30 ago – 1 sep)	Arquitectura FastAPI + interfaz, detección de cara y puntos, tinte inicial, preparación de datos y entrenamiento de YOLO.
2 (16 – 19 sep)	Barba con plantillas, Delaunay propio y *warp*; primera versión de transferencia de pelo.
3 (21 oct – 6 nov)	Física de la barba, gaussianas 2D, juego, espejo 3D, laboratorio, peluca 3D multivista con color y recorte.
4 (9 – 10 nov)	Producto: interfaz Fresh, Docker e integración continua, cámara, motor de cortes, combinación de looks, forma de cara, tinte de barba y YOLO propio en la aplicación.

Tabla 1: Iteraciones del proyecto.

Control de versiones y calidad. El código vive en un repositorio git compartido por los tres integrantes. En las primeras iteraciones cada uno integraba su módulo directamente; desde la

iteración 4, los cambios entran a `main` por *pull requests* con una rama por funcionalidad. GitHub Actions ejecuta las pruebas en cada *push* y en cada *pull request*. Las convenciones de código están escritas en `AGENTS.md`:

- la lógica vive en `backend/app/core`;
- los *endpoints* quedan delgados;
- los datasets y pesos pesados no se versionan, con dos excepciones: los modelos de MediaPipe y `hair_best.pt`, de 6,5 MB.

Verificación de requisitos. Cada requisito se verificó de tres maneras complementarias:

- 47 pruebas automáticas (sección 8.8);
- recorridos completos en el navegador: Chrome sin ventana controlado por el protocolo DevTools, con cámara simulada;
- revisión visual de cuadrículas de resultados sobre 26 caras distintas: piel clara, morena y negra; pelo liso, rizado, largo y corto; con flequillo y con entradas.

Los tiempos se midieron sobre el contenedor (sección 8).

3.3. Organización del equipo

El equipo se dividió por módulos, con un responsable principal por cada uno. La tabla 2 resume los aportes según el historial del repositorio.

Integrante	Aportes principales
Kalos Lazo Mera	Arquitectura del backend y de la API; tinte; preparación de datos y entrenamiento de YOLO; interfaz Fresh; motor de cortes; combinación de looks; forma de cara; tinte de barba; Docker, integración continua y pruebas.
Gianpier Segovia	Barba con plantillas reales; Delaunay de Fortune; *warp* afín; física de la barba; gaussianas 2D; juego de afeitado; espejo 3D; laboratorio (compartido).
Sofía Herrera	Peluca 3D: máscara de transferencia, escaneo con la cámara, peluca de gaussianas multivista, cara 3D, cabeza sin pelo y estudio de color y recorte; laboratorio (compartido).

Tabla 2: Aportes por integrante.

4. Arquitectura del sistema

Fresh es una aplicación cliente–servidor. El mismo proceso de Python sirve la interfaz web y la API, de modo que basta con abrir `http://localhost:8000`. La figura 2 muestra las capas.

4.1. Servidor

El servidor está escrito en Python con FastAPI [27], OpenCV [28] y NumPy [29]. Se organiza en tres niveles:

- **API** (`api/routes.py`): valida parámetros, decodifica la imagen, la reduce a un lado máximo de 2048 px y devuelve un PNG o datos binarios. No contiene lógica gráfica.
- **Orquestación** (`core/pipeline.py`): la clase `StylePipeline` carga los modelos una sola vez y encadena los pasos comunes de cada herramienta: cara, puntos, geometría de la cabeza y probabilidades de segmentación.

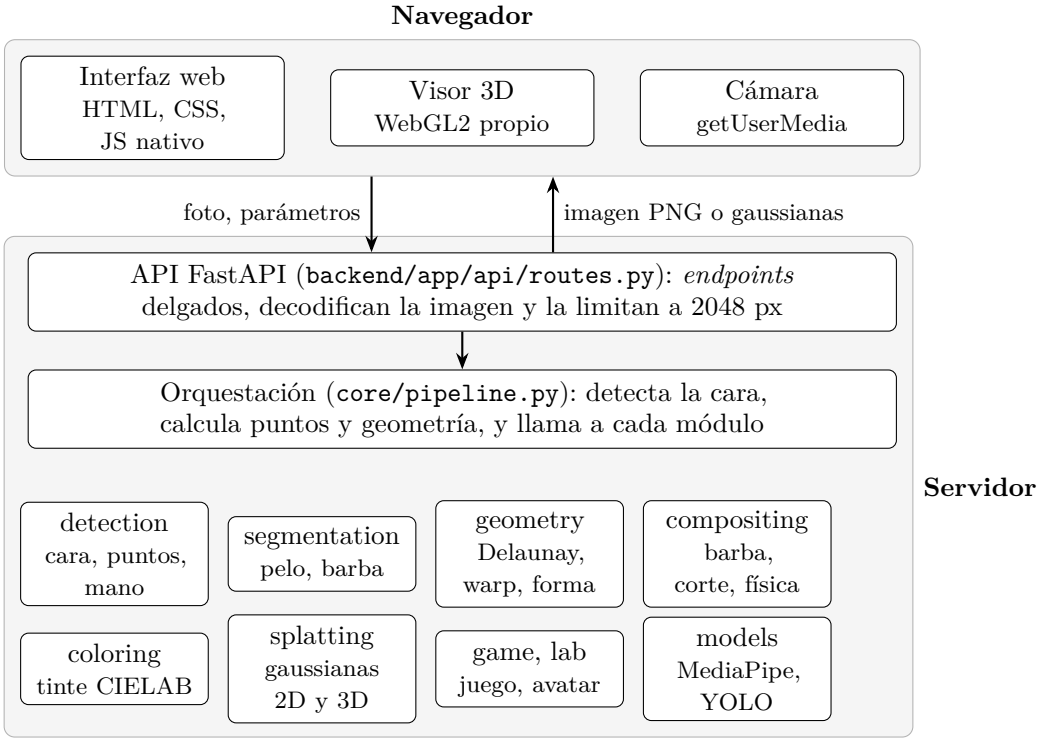


Figura 2: Arquitectura de Fresh. El navegador envía la foto y los parámetros; el servidor devuelve la imagen procesada, o los datos de las gaussianas para que el visor WebGL2 las dibuje.

- **Módulos** (`core/*`): cada técnica vive en su propio paquete, sin depender de la API, y es lo que prueban los tests unitarios.

4.2. Interfaz

La interfaz no usa *frameworks* ni compilación:

- `index.html` define los paneles;
- `app.js` llama a la API de cada herramienta;
- `ui.js` implementa la navegación, la foto compartida, la combinación de looks, la ficha y las sugerencias;
- `splat3d.js` es el visor de gaussianas en WebGL2 [30].

Los datos personales (looks guardados) se quedan en el `localStorage` del navegador.

4.3. Flujo de una petición

Por ejemplo, al aplicar un degradado medio:

1. el navegador envía `POST /api/haircut` con la foto, `style=mid-fade` y la intensidad;
2. el servidor detecta la cara, calcula los 478 puntos y los convierte al formato de 68 puntos, construye la geometría de la cabeza y segmenta el pelo;
3. el módulo `compositing/haircut.py` renderiza el corte sobre un recorte de la cabeza a una resolución de trabajo fija y lo devuelve a la resolución original;
4. la interfaz muestra el resultado en el comparador antes/después.

La lista completa de *endpoints* está en el anexo A.

5. Implementación

Esta sección describe cada módulo:

- qué resuelve y cómo funciona;
- las decisiones que se tomaron y por qué;
- las limitaciones conocidas.

Las rutas de archivos son relativas a `backend/app/core/` salvo que se indique otra cosa.

5.1. Detección de la cara y de los puntos faciales

Cara. En el modo foto, la cara se ubica con el clasificador de Haar de OpenCV [21] (`detection/face_detector.py`) y se elige la de mayor área. En el modo en vivo no se usa, porque falló en el 30 % de 2695 cuadros al girar la cabeza; ahí la cara sale directamente de los puntos.

Puntos. El modelo *Face Landmarker* de MediaPipe [23] devuelve 478 puntos normalizados con profundidad. Este modelo no lo entrenamos: se usa solo para inferencia. Como el resto del sistema se diseñó sobre el estándar iBUG de 68 puntos, `detection/landmarks.py` los convierte así:

- **mandíbula**: los 23 puntos del contorno inferior se remuestrean por longitud de arco a 17 puntos equiespaciados (longitud acumulada e interpolación lineal);
- **cejas, ojos, nariz y boca**: 51 puntos salen de una tabla de correspondencias.

La malla completa con profundidad se conserva para el espejo 3D, el laboratorio y la forma de la cara.

Estabilidad en vivo. En video se usa el modo de seguimiento de MediaPipe y un filtro One-Euro [31]. Es un pasa-bajos cuyo corte crece con la velocidad:

$$f_c = f_{\text{mín}} + \beta\,|\hat{\dot{x}}|, \qquad \alpha = \frac{1}{1+\tau/\Delta t}, \quad \tau = \frac{1}{2\pi f_c}. \tag{5}$$

Con la cara quieta suaviza fuerte y elimina el temblor; con la cara en movimiento sube el corte y no se queda atrás.

Si MediaPipe pierde la cara durante un cuadro, se reutilizan los últimos puntos hasta tres cuadros. Se desplazan con la traslación global estimada por correlación de fase sobre la zona de la cara.

Geometría de la cabeza. A partir de los 68 puntos, `segmentation/transfer_mask.py` calcula:

- el centro entre las orejas a la altura de los ojos;
- el ancho de la cabeza w: la distancia 3D entre orejas, que no se encoge al girar;
- el giro en el plano;
- la altura de las cejas y del mentón;
- el polígono de la cara, formado por la mandíbula y las cejas.

Todos los módulos de pelo trabajan en un marco normalizado (x, y) medido en anchos de cabeza, con origen entre las orejas a la altura de los ojos, y hacia abajo y rotado con la cabeza. Así los parámetros valen igual para una foto de 600 px y para una de 4000 px.

5.2. Segmentación del pelo

El proyecto tiene dos segmentadores de pelo intercambiables.

MediaPipe multiclase. El modelo *selfie multiclass* de MediaPipe (`segmentation/mp_segmenter.py`) da, por píxel, la probabilidad de seis clases: fondo, pelo, piel del cuerpo, piel de la cara, ropa y otros. Se aplica sobre una ventana alrededor de la cabeza a 256×256 px y se reescala. A diferencia de un umbral de oscuridad, distingue el pelo de un fondo oscuro. Las probabilidades de fondo y de piel también se usan en el corte (sección 5.6).

YOLOv8 ajustado por el equipo. Se ajustó un YOLOv8n-seg [25], preentrenado en COCO, para segmentar pelo:

- **datos**: 9450 imágenes de Figaro1k [32] y CelebAMask-HQ [33], convertidas a polígonos con `backend/training/prepare_hair_dataset.py`;
- **entrenamiento**: en Google Colab con `barberstudio_colab.ipynb` (unas 3,5 h en una GPU T4 gratuita);
- **resultado**: mAP50 de máscara de 0,968 en validación;
- **ajuste al borde**: la máscara se ajusta al borde real con un filtro guiado [26] (`segmentation/mask_refiner.py`). Después se descartan los píxeles cuyo color se parece más al anillo de fondo que al interior del pelo.

En el panel de tinte, el usuario elige el modelo. Si elige YOLO, el servidor calcula también la máscara de MediaPipe y devuelve la intersección sobre unión (IoU) entre ambas. Sobre retratos de prueba, la coincidencia va de 0,58 a 0,96 (sección 8).

5.3. Forma de la cara y sugerencias

`geometry/face_shape.py` clasifica la cara en ovalada, redonda, cuadrada, alargada o corazón. Usa la malla 3D, así que las medidas no dependen de la distancia a la cámara. Se divide todo por el ancho de los pómulos $C = \|p_{234} - p_{454}\|$:

$$\text{largo} = \frac{\|p_{10} - p_{152}\|}{C}, \qquad \text{frente} = \frac{\|p_{54} - p_{284}\|}{C}, \qquad \text{mandíbula} = \frac{\|p_{172} - p_{397}\|}{C}, \tag{6}$$

más el ángulo medio del contorno en las esquinas de la mandíbula (puntos 172 y 397, entre la mejilla y el mentón). La figura 3 muestra las medidas sobre una cara.

Las proporciones faciales humanas varían poco [34], así que un umbral fijo clasificaría casi todas las caras en la misma forma. Por eso se calibró con 419 caras de CelebAMask-HQ: cada medida se expresa como desviación z respecto de la media de esa muestra. Las reglas, en orden, son:

- **alargada**: $z_{\text{largo}} > 1$ y $z_{\text{mand}} < 0{,}5$;
- **corazón**: $z_{\text{frente}} > 0{,}6$ y $z_{\text{mand}} < -0{,}6$;
- **cuadrada o redonda**: $z_{\text{mand}} > 0{,}4$ y $z_{\text{largo}} < 0{,}6$; es cuadrada si el ángulo de la mandíbula es cerrado y redonda en caso contrario;
- **redonda** también si $z_{\text{largo}} < -0{,}9$;
- **ovalada** en cualquier otro caso.

En la muestra, la forma más frecuente es la ovalada (37 %). El 73 % de las caras con el atributo "rellena" del dataset quedan como redondas o cuadradas.

Cada forma lleva una recomendación de cortes y barbas, basada en la práctica de barbería: alargar una cara redonda con lados cortos, ensanchar una cara alargada con la barba completa, etc. La interfaz las marca como "Te queda". Es una heurística: no es una regla médica ni estética universal.

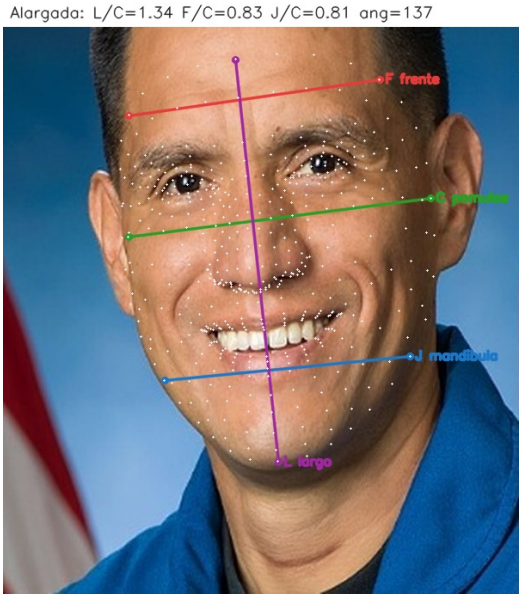


Figura 3: Medidas sobre la malla 3D: frente (F), pómulos (C), mandíbula (J) y largo (L). Arriba, las proporciones y la forma resultante.

5.4. Barba con plantillas reales

La barba no se dibuja desde cero: se ajusta una plantilla real a la cara de la persona.

Plantillas. `backend/training/harvest_beards.py` recorre CelebAMask-HQ una sola vez y procesa cada foto así:

1. filtra por atributos: hombre, sin sombrero, nítida, y clasifica en barba completa, candado, bigote o tres días;
2. detecta los 68 puntos de la cara;
3. calcula por píxel un alfa de pelo, "más oscuro que su entorno": $\alpha = \mathrm{clip}\left((Y_{\mathrm{local}} - 10 - Y)/14, 0, 1\right)$;
4. descarta las plantillas asimétricas o con poco contraste;
5. guarda el recorte BGRA, sus puntos y el tono de piel en YCrCb.

Quedan 10 plantillas. En tiempo de ejecución se elige, dentro del estilo pedido, la de croma de piel más cercano al del usuario.

Malla. A los 68 puntos se suman 15 puntos virtuales bajo la mandíbula, para que la barba bajo el mentón quede dentro de la malla. Sobre los 83 puntos se calcula la triangulación de Delaunay con nuestra implementación del algoritmo de Fortune (`geometry/delaunay_fortune.py`):

- una cola de prioridad (`heapq`) guarda los eventos de sitio y de círculo;
- cada evento de círculo aporta directamente un triángulo, sin construir el diagrama de Voronoi completo.

También hay una implementación de Bowyer–Watson (`geometry/delaunay.py`) que da el mismo resultado; se usó para validar.

***Warp* afín por triángulo.** `geometry/warp.py` reemplaza a las funciones de OpenCV por una versión propia. Para cada triángulo:

1. resuelve el sistema 3×3 de la afín inversa (destino a origen);
2. recorre los píxeles del rectángulo envolvente del triángulo de destino y se queda con los que tienen coordenadas baricéntricas no negativas;
3. lee el origen con interpolación bilineal.

Solo se procesan los triángulos cercanos a la barba.

Color. El color del pelo se mide como la mediana dentro de la máscara de pelo y se pasa a YCrCb. A la nube de color de la barba deformada se le suma un desplazamiento constante hacia ese color. Un desplazamiento, y no un reemplazo, conserva la variación interna, es decir, los pelos. El croma se limita a tonos naturales para que un fondo azul no tiña la barba.

Región y borde. La región de cada estilo se arma con polígonos y elipses sobre los 68 puntos:

- el bigote es una banda en arco que sigue una curva de Bézier cuadrática;
- a la región se le restan los labios y se recorta al casco convexo de la cara;
- el borde se suaviza con un *feather* orgánico: distancia al borde (*distance transform*) más ruido.

Composición en tres capas. Sea ρ la densidad elegida y α_r el alfa de la región:

1. **cuerpo**: *multiply* con $m = 0{,}35 + 0{,}65\, c/255$ y ruido anisótropo orientado según la dirección del pelo. La luz de la piel sigue viéndose a través de la barba;
2. **hebras**: curvas de Bézier cuadráticas que siguen un campo de dirección hacia abajo y hacia afuera. Su cantidad es proporcional a ρ, hasta 9000;
3. **detalle**: se transfiere solo la alta frecuencia de la plantilla, plantilla $- G_\sigma$(plantilla). Así se conservan los pelos reales sin traer el brillo ni el color de la otra persona. Se descartó un *Poisson blending* [35] porque igualaba el brillo con el borde y aclaraba la barba.

El estilo "tres días" reemplaza las capas 2 y 3 por un grano de 1 px cuyo umbral baja al subir la densidad. La figura 4 muestra los cuatro estilos.

En vivo. La textura se genera una vez y se ancla a la malla de referencia (`LiveAnchor`):

- en cada cuadro, los ruidos se deforman con el mismo *warp*;
- cada hebra guarda sus coordenadas baricéntricas, de modo que queda pegada a la piel en lugar de "nadar" sobre la imagen;
- el contraste del grano se restaura después del *warp*, porque interpolar ruido blanco reduce su varianza.

5.5. Tinte en CIELAB

El tinte (`coloring/dye.py`) recibe una máscara suave del pelo $\alpha \in [0, 1]$ y un color objetivo con componentes (L_t, a_t, b_t) en CIELAB.

Por qué CIELAB. En una primera versión se cambiaba el tono en HSV. Eso conserva la oscuridad del pelo, así que un pelo negro nunca podía quedar rubio. En CIELAB la luminosidad está separada del color, así que se puede mover la luminosidad base hacia la del tinte sin perder el detalle de cada mechón.



Figura 4: Original y los cuatro estilos de barba sobre un retrato de dominio público: completa, candado, bigote y tres días.

Paso 1: luminosidad. Sean $\bar{L}$ y s_L la media y la desviación de L^* dentro del pelo, y G_σ un desenfoque gaussiano con $\sigma = 1{,}5$ px. La imagen se descompone en sombreado (variación lenta) y detalle (mechones):

$$\text{detalle} = L - G_\sigma(L), \qquad \text{sombreado} = G_\sigma(L) - \bar{L}, \tag{7}$$

$$L' = \underbrace{0{,}72\,L_t + 0{,}28\,\bar{L}}_{\text{base}} + k\,(0{,}8\,\text{sombreado} + 1{,}2\,\text{detalle}), \qquad k = \text{clip}(20/s_L,\ 0{,}9,\ 1{,}8). \tag{8}$$

El factor k normaliza el contraste: un pelo de contraste bajo, como uno negro liso, gana detalle, y uno muy contrastado no se exagera.

Paso 2: color. El croma se toma del tinte y se atenúa en las sombras más profundas, como ocurre con un pigmento real:

$$a' = 128 + (a_t - 128)\,(0{,}4 + 0{,}5\,d), \qquad d = \text{clip}\left(\frac{L'}{\text{máx}(L_t, 40)}, 0, 1\right), \tag{9}$$

y lo mismo para b'.

Paso 3: mezcla. El resultado se mezcla con la imagen original según α multiplicado por la intensidad elegida.

Máscara. La máscara viene de MediaPipe o del YOLO propio (sección 5.2). Su borde se suaviza hacia adentro, para no teñir la piel ni el fondo.

5.6. Motor de cortes de pelo

El corte es la pieza más compleja de la interfaz de estilos (`compositing/haircut.py`). Para cortar el pelo hay que resolver cuatro problemas:

1. quitar el pelo que sobra y reconstruir lo que había detrás;
2. saber dónde termina realmente la cabeza;
3. ubicar la línea del pelo aunque haya flequillo;
4. hacer crecer pelo corto creíble, con la luz de la foto.

Resolución de trabajo. El corte se calcula sobre un recorte de la cabeza escalado para que el ancho de la cabeza mida 420 px. Al volver a la resolución original se suma de nuevo el detalle de alta frecuencia de la foto en las zonas que no cambiaron. Así una foto de 4000 px tarda lo mismo que una de 800 px y no pierde nitidez.

Línea del pelo. Primero se busca la frente visible: desde las cejas se sube por la columna central mientras la probabilidad de piel de la cara sea alta.

- Si se ven al menos $0{,}2\,w$ de frente, el borde de esa piel es la línea del pelo real.
- Si no, hay flequillo, y se asume una frente típica de $0{,}30\,w$ sobre las cejas, en línea con las proporciones faciales de [34].

Hacia los costados, la línea se retrae un poco en las sienes y baja hasta la patilla con una esquina redondeada (un cuarto de circunferencia). Se le añade un ruido de baja frecuencia para que no sea una línea perfecta.

Cráneo. La cabeza rapada es una superelipse de exponente 2,15 en el marco (x, y):

$$\left|\frac{x}{0{,}55}\right|^{2{,}15} + \left|\frac{\min(y, 0)}{y_{\text{tope}}}\right|^{2{,}15} \leq 1. \tag{10}$$

Sus costados bajan hasta las orejas. La altura y_{tope} se ajusta a cada persona: es la mayor entre la línea del pelo menos $0{,}33\,w$ y la coronilla medida (el punto más alto de pelo o piel en la columna central) más $0{,}03\,w$. Si el pelo ya es corto, el cráneo no sobresale de la silueta real. Por último, el cráneo se intersecta con la silueta de la persona, ligeramente dilatada.

Relleno del fondo con *pull-push*. El pelo que queda fuera del cráneo hay que quitarlo. El primer intento usó *inpainting* de Telea [18], que propaga el color de todo el borde del hueco; como ese borde incluye pelo, arrastraba el pelo hacia el fondo y dejaba "pelo fantasma". La solución es *pull-push* [19] con una máscara de píxeles conocidos elegida a mano:

- cuentan solo los píxeles de fondo seguro ($p_{\text{fondo}} > 0{,}6$) o de ropa, fuera del pelo;
- cerca del pelo se exige $p_{\text{fondo}} > 0{,}9$.

En cada nivel de la pirámide, el color es $\sum w\,c / \sum w$, y en la fase *push* se mezcla con el nivel grueso según el peso, $\text{clip}(2w, 0, 1)$. La figura 5 compara ambos métodos.

Después del relleno se buscan **mechones sueltos**: píxeles cerca del hueco que se diferencian del fondo suavizado en más de 30 niveles. Se agregan al hueco y se rellena de nuevo. Además, el fondo rellenado recibe un ruido cuya amplitud es la energía de alta frecuencia local del fondo, propagada también con *pull-push*, para que no se vea más liso que el resto.



Figura 5: Rellenar el fondo donde se quita pelo. Telea toma color de todo el borde y arrastra el pelo; *pull-push* con solo píxeles de fondo deja un relleno limpio.

Piel bajo el pelo. Dentro del cráneo, la piel que tapaba el pelo (cuero cabelludo o frente bajo un flequillo) se rellena con *pull-push*. Como fuente se usan solo píxeles de piel de la cara alejados del pelo, porque la piel pegada a la línea del pelo suele estar sombreada y dejaba una marca. La transición hacia la piel real es gradual.

Pelo corto. El pelo nuevo tiene el color del propio pelo: la mediana en CIELAB del 45 % más oscuro de los píxeles de pelo. Se sombrea con un modelo de Lambert [5]:

- la normal sale de tratar el cráneo como un elipsoide;

- la dirección de la luz se estima de la asimetría de brillo entre las dos mitades de la cara, más una componente desde arriba;
- la luminosidad final es $L = L_{\text{pelo}}\,(0{,}55 + 0{,}6\,s)$, donde $s = 0{,}62 + 0{,}38\,\text{máx}(0, \hat{n} \cdot \hat{l})$ es el sombreado.

La textura combina dos ruidos: uno anisótropo (desenfoque gaussiano mucho más largo en vertical que en horizontal) que imita trazos cortos, y uno de baja frecuencia para variaciones de densidad. La cobertura del pelo corto es

$$c = \rho(x, y) \cdot z_{\text{pelo}} \cdot h_{\text{cráneo}} \cdot (1 + 0{,}08\,n_{\text{trazo}} + 0{,}06\,n_{\text{parche}}), \tag{11}$$

donde z_{pelo} vale 1 por encima de la línea del pelo y ρ es la densidad del estilo. La densidad base depende de la intensidad elegida t: $\rho_0 = 0{,}92\,(1{,}1 - 0{,}4\,t)$. Más intensidad significa pelo más corto, con más cuero cabelludo visible.

Estilos. Todos los cortes se definen sobre el mismo marco. La parte de arriba se conserva dentro de una franja $|x| < r(y)$, con

$$r(y) = 0{,}36 + 0{,}16\ \text{smoothstep}\Big(\frac{-0{,}40 - y}{0{,}25}\Big), \tag{12}$$

que se ensancha hacia la coronilla. En los costados, la densidad sube desde la piel en y_{piel} hasta pelo corto en y_{corto}:

$$\rho(y) = \rho_0\ \text{mín}\Big(f + (1 - f)\sqrt{\text{smoothstep}\,(\tfrac{y_{\text{piel}} - y}{y_{\text{piel}} - y_{\text{corto}}})},\ c_{\text{máx}}\Big), \tag{13}$$

donde f es la densidad mínima y $c_{\text{máx}}$, la máxima. La tabla 3 da los parámetros.

Corte	y_{piel}	y_{corto}	Particularidad
Degradado bajo	0,10	−0,12	Piel solo junto a la oreja.
Degradado medio	−0,08	−0,30	Piel hasta la sien.
Degradado alto	−0,24	−0,46	Piel hasta lo alto de los costados.
Taper	0,24	0,00	Nunca llega a piel: $f = 0{,}35$.
Undercut	–	–	Costados planos y muy cortos ($c_{\text{máx}} = 0{,}22$) con borde duro (transición de $0{,}02\,w$).
Raya marcada	−0,08	−0,30	Degradado medio más una línea afeitada en $x = r(y) - 0{,}09$ sobre el cuero cabelludo de un lado.
Crew cut	–	–	Todo corto, más denso arriba; conserva el 30 % de la textura original arriba y sube el tope del cráneo $0{,}035\,w$.
Rapado	–	–	Pelo corto uniforme por encima de la línea del pelo.

Tabla 3: Parámetros de cada corte en el marco de la cabeza (unidades de ancho de cabeza, y hacia abajo).

Protección de la cara. El polígono de la cara, contraído un 5 % hacia su centro y erosionado, nunca se modifica; las pruebas automáticas lo verifican para los ocho cortes. La contracción deja libres las patillas del borde, que se degradan junto con los costados.

5.7. Tinte también en la barba

Para teñir la barba hay que encontrarla, y el segmentador no la marca como pelo. `segmentation/facial_hair.py` trabaja dentro de la zona donde crece una barba completa, recortada en el borde de la mandíbula. Un píxel cuenta como barba si cumple tres condiciones:

1. es **más oscuro** que la piel de la propia persona: smoothstep$((L_{\text{piel}} - L - 10)/25)$, donde L_{piel} se mide en las mejillas, sobre la nariz;

2. tiene **textura de pelo**: desviación local de L alta en una ventana de $0{,}012\,w$;
3. es **menos rojo** que la piel, porque el pelo tiene menos a^*.

Las sombras lisas no cumplen la condición de textura. El borde largo de la mandíbula sí la cumple, pero se descarta con la coherencia del tensor de estructura [20]: un borde recto tiene coherencia cercana a 1, mientras que la barba tiene trazos en todas direcciones. Finalmente, las zonas oscuras encerradas por barba detectada (el centro de un candado negro, que no tiene textura) se incorporan con un cierre morfológico.

Probado sobre 10 caras, las cinco sin barba quedan limpias y los bigotes se detectan completos. Cuando la barba la puso la propia aplicación (sección 5.8), su estilo se conoce y se usa directamente la máscara de ese estilo.

5.8. Combinación de looks y ficha para el barbero

Bajo cada resultado, la interfaz ofrece aplicar otra herramienta sobre él. Por ejemplo:

1. primero la barba;
2. luego el tinte con barba;
3. al final el corte (figura 6).

El resultado de cada paso pasa a ser la foto de la herramienta siguiente. El comparador sigue mostrando la foto original como "antes". El tinte, al recibir una foto con barba hecha por la aplicación, recibe también el estilo de esa barba. El corte toma el color del pelo ya teñido.

El look guardado lleva el nombre de todos los pasos. La ficha para el barbero (figura 17) lista cada servicio con una indicación práctica:

- el número de máquina o de peine;
- dónde va cada número;
- la referencia hexadecimal del color.

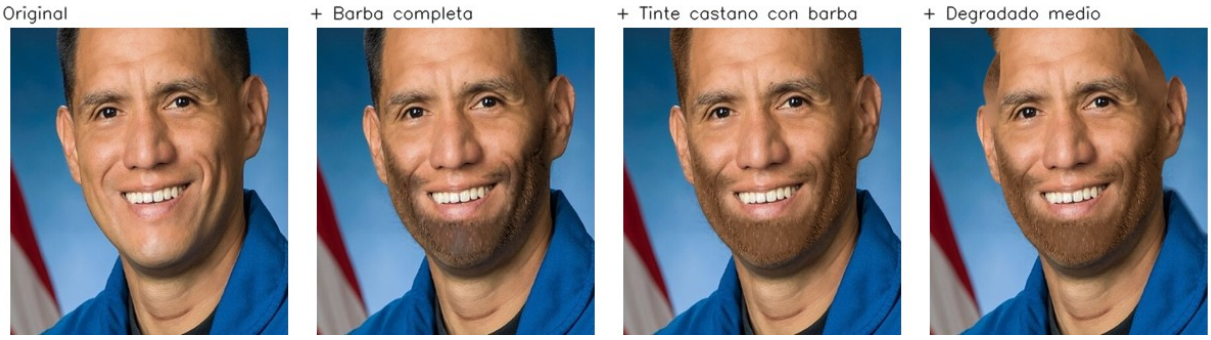


Figura 6: Look combinado sobre el retrato de ejemplo: barba completa, tinte castaño que incluye la barba y degradado medio.

5.9. Física de la barba

En el modo en vivo, la barba larga se simula con física (`compositing/beard_physics.py`).

Modelo. Cada hebra es una cadena de cuatro partículas, y hay una hebra por cada 30 píxeles de la región (hasta 2600):

- la raíz se guarda en coordenadas baricéntricas de la malla de referencia, así que sigue a la piel en cada cuadro;
- la pose de reposo crece en la dirección del pelo y gira con la cabeza;
- el largo depende de cuánto está la raíz por debajo de la boca;
- las raíces del labio superior crecen hacia los costados, para que no cuelguen sobre los dientes.

Integración. En cada cuadro se dan cuatro subpasos de longitud $h = \Delta t/4$ con Verlet amortiguado [15]:

$$v = 0{,}86\,(x - x_{\text{prev}}), \qquad x \leftarrow x + v + g\,h^2, \qquad x \leftarrow x + k_i\,(x_{\text{reposo}} - x), \tag{14}$$

con $g = 2{,}2\,w$ px/s^2 y una rigidez $k_i = [0;\ 0{,}34;\ 0{,}22;\ 0{,}13]$ que baja hacia la punta: la base sigue a la cara y la punta tiene inercia. Después, dos iteraciones de restricción de distancia (PBD [16, 17]) mantienen el largo de cada tramo. Por eso la hebra se balancea como un péndulo y no se estira. La raíz se interpola entre el cuadro anterior y el actual para que no salte.

Render. Cada hebra se suaviza con un corte de esquinas de Chaikin [36], de 4 a 8 puntos. Cada tramo se dibuja como una gaussiana 2D alargada, con σ a lo largo igual a medio tramo y de 0,5 a 0,75 px a lo ancho. Las gaussianas se acumulan sin ordenar, con densidad $A = \sum o_i G_i$ y cobertura $\alpha = 1 - e^{-A}$, como en la ley de Beer–Lambert, de modo que muchos pelos superpuestos saturan suavemente. Unos 2000 tramos cuestan unos 10 ms, vectorizados sobre parches de 13×13.

En una secuencia con vaivén y un tirón brusco, el retraso de las puntas llega a entre 8 y 12 px y se recupera solo. La correlación entre la velocidad de la cabeza y el retraso es de 0,78 a 0,82 (figura 7).



Figura 7: Barba con física: en reposo y durante un movimiento rápido de la cabeza. Las puntas se atrasan y luego vuelven.

5.10. Gaussianas 2D: "¿Cuántas gaussianas eres?"

`splatting/gaussian_image.py` reconstruye un retrato con N gaussianas 2D, al estilo de GaussianImage [14], sin bibliotecas de diferenciación automática.

Representación. Cada gaussiana tiene un centro μ_i, un peso RGB c_i y el factor de Cholesky de la inversa de su covarianza, $L = \left(\begin{smallmatrix} l_1 & 0 \\ l_2 & l_3 \end{smallmatrix}\right)$. Con $\Sigma^{-1} = LL^\top$, la covarianza es siempre definida positiva. Con $u = l_1 d_x + l_2 d_y$ y $v = l_3 d_y$:

$$G_i(x) = \exp\!\left(-\tfrac{1}{2}(u^2 + v^2)\right), \qquad C(x) = \sum_i c_i\, G_i(x). \tag{15}$$

Optimización. Se minimiza el error cuadrático medio contra la imagen, usando gradientes analíticos:

- respecto de la forma: $\partial q/\partial l_1 = 2u\,d_x$, $\partial q/\partial l_2 = 2u\,d_y$ y $\partial q/\partial l_3 = 2v\,d_y$;
- respecto del centro: $\partial q/\partial \mu_x = -2u\,l_1$ y $\partial q/\partial \mu_y = -2u\,l_2 - 2v\,l_3$.

Los parámetros se actualizan con Adam [37] escrito a mano, en 110 a 160 iteraciones.

Detalles de implementación.

- La inicialización coloca más gaussianas donde hay bordes: muestrea con probabilidad proporcional a $0{,}35 + |\nabla I|$.
- Cada gaussiana se evalúa solo en un parche de radio $\lceil 2{,}5\,\sigma_{\text{máx}} \rceil$, y los parches se agrupan por radio. Así no hay bucles por gaussiana.
- La acumulación usa `np.bincount`. Una etapa pasó de 10–21 s a 1–4 s.

La interfaz lo convierte en un juego: el retrato aparece con 20, 60, 200, 600 y 2000 gaussianas, y hay que reconocer la cara lo antes posible (figura 8).

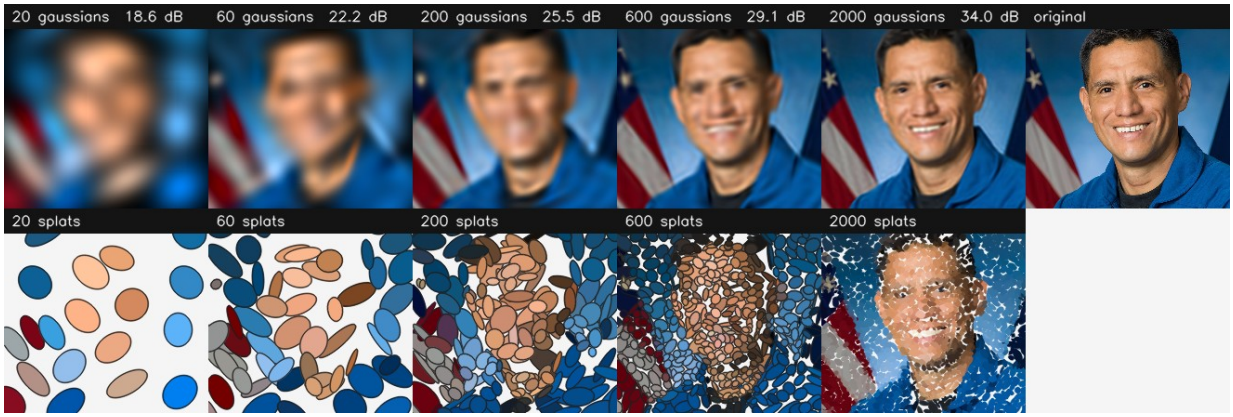


Figura 8: Retrato reconstruido con 20, 60, 200, 600 y 2000 gaussianas 2D junto al original (arriba), y cada gaussiana dibujada como una elipse (abajo).

5.11. Espejo 3D con *Gaussian Splatting*

El espejo convierte una foto en una cabeza de unas 100 000 gaussianas 3D que se puede girar. Implementa, desde cero, el paso de *render* de 3DGS [12]. Las gaussianas no se entrenan desde varias vistas: se construyen a partir de una sola foto y de la malla 3D de MediaPipe.

Construcción (`splatting/head3d.py`).

1. La malla de 478 puntos se triangula con el Delaunay propio en (x, y) y se eleva a 3D con la profundidad estimada.
2. **Cara**: cada triángulo recibe gaussianas en proporción a su área 3D (no a la proyectada), con muestreo estratificado. Cada gaussiana es un disco tangente, $\Sigma = R\,\mathrm{diag}(s^2, s^2, (0{,}4s)^2)R^\top$, donde R es el marco del triángulo. Su color se lee de la foto con interpolación bilineal.
3. **Barba**: cada hebra son tres gaussianas alargadas que crecen hacia afuera y hacia abajo desde raíces ancladas con coordenadas baricéntricas.
4. **Resto de la foto** (pelo, orejas, fondo): un relieve cuya profundidad se acerca a la del contorno de la cara más cercano, con caída exponencial.

Visor (`frontend/js/splat3d.js`, WebGL2). Para cada gaussiana:

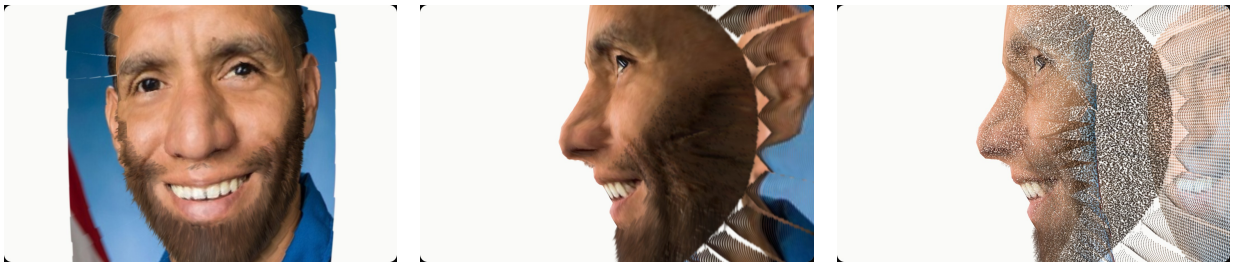
- el *vertex shader* proyecta la covarianza con EWA, $\Sigma_{2D} = JW\Sigma W^\top J^\top + 0{,}3\,I$ [13], calcula autovalores y autovectores en forma cerrada y dibuja un cuadrilátero orientado de $\pm 3\sigma$;

- el *fragment shader* evalúa $\alpha = a\, e^{-r^2/2}$.

Las gaussianas se ordenan de atrás hacia adelante con un *counting sort* de 16 bits, en tiempo $O(N)$, y se mezclan con `SRC_ALPHA, ONE_MINUS_SRC_ALPHA`. La opción "Ver cada gaussiana" achica la huella para mostrar la técnica (figura 9).

Exportación. Se exporta al formato `.ply` de 3DGS, compatible con visores externos:

- la covarianza se descompone en escala y cuaternión;
- el color pasa a armónicos esféricos de grado 0, $f = (c - 0{,}5)/0{,}2821$;
- la opacidad pasa a *logit*.



(a) Frente **(b)** Girada **(c)** Cada gaussiana

Figura 9: Espejo 3D con barba completa, construido desde una sola foto. Al girar mucho se ven los límites de reconstruir desde una vista: el relieve detrás de la cara se estira.

5.12. Peluca 3D multivista

La peluca 3D escanea el peinado de una persona con la cámara y lo convierte en una peluca de gaussianas. Esa peluca se puede ver sobre un maniquí, sobre la cara escaneada o sobre otra persona en vivo (`splatting/hair3d.py`, `splatting/face3d.py`, `compositing/bald.py`).

Escaneo. Durante 12 segundos la persona gira la cabeza unos 180°, y se guardan capturas con el ángulo de giro estimado. En cada captura, la máscara de pelo se obtiene combinando el segmentador, la distancia de color al pelo y una elipse de la cabeza (`segmentation/transfer_mask.py`).

Forma: *visual hull* por cortes. Para cada altura y cada dirección horizontal (360 direcciones), cada captura indica, desde su ángulo, dónde empieza y dónde termina el pelo. Intersecar esas franjas da el corte transversal real de la peluca, es decir, la envolvente visual [38], calculada rebanada por rebanada. Las direcciones que ninguna captura vio siguen una forma de cabeza:

- tan ancha como la vista más frontal;
- nunca delante de la cara;
- redondeada sobre el cráneo y más plana donde cuelga pelo largo.

Textura. Cada punto de la superficie toma color de las capturas que mejor lo ven, mezcladas según qué tan de frente lo ven, para que no haya costuras. La nuca nunca se ve: recibe el color del último pelo que vio cada perfil, con estrías verticales. Cada gaussiana se alarga en la dirección del pelo, estimada con el tensor de estructura (perpendicular al gradiente dominante; hacia abajo donde no hay orientación clara).

Estudio. En el navegador, la peluca se puede editar de tres formas:

- **color**: se recolorean las gaussianas con la intensidad elegida;
- **largo**: se eliminan las gaussianas por debajo de una altura;

- **tijeras**: un pincel en pantalla corta las gaussianas más cercanas a la cámara dentro de su radio, usando la proyección de cada gaussiana; los cortes se pueden deshacer.

Prueba en vivo. Para probar la peluca sobre otra persona, el navegador sigue la cabeza con MediaPipe en JavaScript y el servidor quita el pelo propio de esa persona. El fondo se aprende de cuadros anteriores o se rellena, y el cuero cabelludo se sombrea a partir de la piel de la cara. La figura 10 muestra una peluca construida desde una sola foto, el caso mínimo; con el escaneo de 180° se cierra también la parte de atrás.



Figura 10: Peluca de gaussianas sobre el maniquí, construida desde un único retrato frontal.

5.13. Laboratorio: avatar en vivo

El laboratorio (`lab/avatar.py`) escanea la cabeza desde varios ángulos y construye un avatar de gaussianas. Cada gaussiana guarda su triángulo de la malla, sus coordenadas baricéntricas, un desplazamiento y una covarianza expresados en el marco local del triángulo, cuyos ejes son dos aristas y la normal.

En cada cuadro de la cámara, la malla nueva reconstruye el marco de cada triángulo y la gaussiana lo sigue. Por eso el avatar copia los gestos de la persona y puede verse de perfil. Está marcado como experimental en la interfaz.

5.14. Contrarreloj: el juego de afeitado

La persona tiene 30 segundos para afeitar una barba con física hasta dejar un estilo objetivo: candado, bigote o collar (`game/speedrun.py`). La "máquina" puede ser el mouse o la punta del dedo índice, detectada con el modelo de mano de MediaPipe; corta solo con el índice extendido y el dedo medio doblado.

Detección del corte. A 10 cuadros por segundo, el dedo salta decenas de píxeles entre cuadros, así que probar si toca un punto dejaría pelos sin cortar. En su lugar, se cortan las hebras cuya raíz o primer nodo está a menos de $r = 0{,}05\,w$ del segmento recorrido $\overline{p_0 p_1}$:

$$t = \operatorname{clamp}\left(\frac{(x - p_0)\cdot(p_1 - p_0)}{\|p_1 - p_0\|^2}, 0, 1\right), \qquad d = \|x - (p_0 + t(p_1 - p_0))\|. \tag{16}$$

Las hebras cortadas caen con física y se desvanecen.

Puntaje. Se calcula como

$$1000 \cdot \text{limpieza} \cdot \text{conservación}, \tag{17}$$

donde la limpieza es la fracción de pelo afeitado fuera del objetivo y la conservación, la fracción de pelo que queda dentro de él. Se descartó la IoU como puntaje porque, sin tocar nada, ya daba 0,72 para el candado.



Figura 11: Una ronda de Contrarreloj con objetivo "candado", de la barba completa al estilo final.

6. Herramientas utilizadas

La tabla 4 indica qué herramienta resolvió cada parte del proyecto. Donde dice "propio", el algoritmo está escrito por el equipo sobre operaciones básicas de NumPy.

Herramienta	Qué resolvió
Python 3.12 + NumPy [29]	Lenguaje del servidor y operaciones vectorizadas sobre imágenes. Base de todos los algoritmos propios.
FastAPI [27] + Uvicorn	Servidor HTTP y API; también sirve la interfaz web.
OpenCV [28]	Lectura y escritura de imágenes, conversiones de color, desenfoques, morfología, *distance transform*, clasificador de Haar, filtro guiado e *inpainting* de Telea (solo como referencia de comparación).
MediaPipe [22]	Modelos preentrenados, usados solo para inferencia: malla facial de 478 puntos, segmentador multiclase y puntos de la mano. También corre en el navegador para seguir la cabeza en la prueba en vivo de la peluca.
Ultralytics YOLOv8 + PyTorch [25, 39]	Ajuste fino y ejecución del segmentador de pelo propio (`hair_best.pt`). Es opcional: sin él, la aplicación usa MediaPipe.
Google Colab	Entrenamiento de YOLO con GPU (`backend/training/barberstudio_colab.ipynb`).
WebGL2 [30]	Visor de gaussianas 3D en el navegador (*shaders* propios).
HTML, CSS y JavaScript nativos	Interfaz sin *frameworks* ni compilación.
pytest + GitHub Actions	47 pruebas automáticas que se ejecutan en cada cambio.
Docker	Imagen reproducible con todas las dependencias y los modelos.
LaTeX	Este documento.

Tabla 4: Herramientas y la parte del proyecto que resolvió cada una.

Los efectos animados de la interfaz (texto que rota, títulos que aparecen, franja de líneas) son versiones en JavaScript nativo de componentes de React Bits, con licencia MIT y *Commons Clause*. Los retratos de prueba de las figuras son retratos oficiales de la NASA, de dominio público.

7. Cómo reproducir el proyecto

Esta sección permite instalar y ejecutar Fresh desde cero. El código fuente completo se entrega en la carpeta `codigo-fuente/`, que contiene lo mismo que la raíz del repositorio. Los comandos se escriben en una terminal (macOS, Linux o Windows con WSL), situada en esa carpeta.

Pieza	Origen
Detección de cara (Haar), 478 puntos faciales, segmentación base, puntos de la mano	Biblioteca o modelo
Segmentador de pelo YOLOv8	Arquitectura de biblioteca, ajuste fino propio
Triangulación de Delaunay (Fortune y Bowyer–Watson)	Propio
Warp afín, coordenadas baricéntricas, interpolación bilineal	Propio
Composición de la barba, hebras, *feather*, anclaje a la malla	Propio
Tinte en CIELAB, tinte de barba, forma de la cara	Propio
Motor de cortes (*pull-push*, cráneo, línea del pelo, sombreado)	Propio
Física de hebras (Verlet + PBD) y juego	Propio
Gaussianas 2D (gradientes analíticos, Adam)	Propio
Gaussianas 3D, proyección EWA, ordenamiento, exportación `.ply`	Propio
Peluca multivista (*visual hull* por cortes, textura, estudio)	Propio

Tabla 5: Qué viene de bibliotecas y qué implementó el equipo.

7.1. Opción A: con Docker (recomendada)

Es la forma más simple, porque instala todas las dependencias y los modelos dentro de un contenedor.

1. Instalar Docker Desktop desde `https://www.docker.com` y abrirlo.
2. Construir la imagen y levantar la aplicación (la primera vez tarda varios minutos porque descarga PyTorch):

   ```
   docker compose up -d --build app
   ```

3. Esperar a que el contenedor esté sano y abrir `http://localhost:8000` en Chrome:

   ```
   docker compose ps
   ```

4. Para detenerla:

   ```
   docker compose down
   ```

7.2. Opción B: con Python

1. Instalar Python 3.12 desde `https://www.python.org`.
2. Crear un entorno virtual e instalar las dependencias:

   ```
   cd backend
   python3 -m venv venv
   source venv/bin/activate
   pip install -r requirements.txt -r requirements-dev.txt
   ```

3. Opcional, para usar el YOLO propio:

   ```
   pip install -r requirements-yolo.txt
   pip uninstall -y opencv-python
   pip install --force-reinstall --no-deps "opencv-contrib-python>=4.9,<5"
   ```

   Ultralytics instala `opencv-python`, que pisa a `opencv-contrib-python` y elimina los módulos `cv2.face` y `cv2.ximgproc`. Las dos últimas líneas lo corrigen.

4. Iniciar el servidor desde la carpeta `backend`:

   ```
   uvicorn app.main:app --port 8000
   ```

   O, desde la raíz, `./run.sh`, que hace los pasos 2 y 4 automáticamente.

5. Abrir `http://localhost:8000`.

7.3. Modelos

Todos los modelos necesarios están en `backend/models/`:

- `face_landmarker.task`: malla facial de MediaPipe.
- `selfie_multiclass_256x256.tflite`: segmentador multiclase de MediaPipe.
- `hand_landmarker.task`: puntos de la mano, para el juego.
- `lbfmodel.yaml`: puntos de respaldo de OpenCV.
- `hair_best.pt`: el YOLO de pelo entrenado por el equipo.

Si falta `hair_best.pt` o no está instalado Ultralytics, la aplicación usa MediaPipe y el selector de modelo no aparece. La interfaz carga dos recursos desde internet: las fuentes tipográficas, que tienen respaldo local si no cargan, y la versión en JavaScript de MediaPipe, que solo usa la prueba en vivo de la peluca.

7.4. Pruebas

Desde `backend`, con el entorno activado:

```
python -m pytest tests -q
```

El resultado esperado es `47 passed`.

7.5. Reentrenar el segmentador de pelo

1. Descargar Figaro1k [32] y CelebAMask-HQ [33] en `data/`. Las páginas de los datasets indican cómo pedir acceso; no se incluyen por tamaño y licencia.
2. Convertir las máscaras a polígonos de YOLO:

   ```
   python backend/training/prepare_hair_dataset.py
   ```

3. Abrir `backend/training/barberstudio_colab.ipynb` en Google Colab, elegir una GPU T4 y ejecutar las celdas en orden. Si hay GPU local, se puede usar en su lugar:

   ```
   python backend/training/train_hair.py --epochs 60 --device cuda
   ```

4. Copiar el mejor peso resultante a `backend/models/hair_best.pt`.

7.6. Usar la aplicación

1. En *Inicio*, subir una foto, tomarla con la cámara o usar la foto de ejemplo. Conviene una foto de frente, con luz pareja y sin gorra.
2. En *Estilos*, elegir *Barba*, *Tinte* o *Corte*, ajustar el estilo y la intensidad, y aplicar.

3. Bajo el resultado, usar “Agregar tinte”, “Agregar corte” o “Agregar barba” para combinarlos.
4. Guardar el look o descargar la ficha para el barbero.
5. En *3D*, construir la cabeza en el *Espejo 3D* o escanear una peluca en *Peluca 3D*.
6. En *Juegos*, jugar Contrarreloj con la cámara o adivinar el retrato en *Gaussianas*.

8. Resultados

Todas las figuras de esta sección usan retratos oficiales de la NASA, de dominio público. Durante el desarrollo, los cortes y el tinte de barba se revisaron también sobre 19 caras variadas de CelebAMask-HQ, que no se reproducen aquí por licencia.

8.1. Cortes de pelo

La figura 12 muestra los ocho cortes sobre tres personas con pelo distinto: liso y oscuro, afro, y castaño claro. En todos los casos el fondo detrás del pelo quitado queda limpio, sin pelo fantasma, y la cara no cambia. Cada degradado muestra la piel a una altura distinta:

- el *taper* conserva pelo corto hasta abajo;
- el *undercut* afeita los costados de forma plana bajo un borde duro;
- la raya marcada agrega la línea afeitada entre la parte de arriba y el costado;
- el *crew cut* y el rapado reemplazan todo el volumen por pelo corto del color de la persona, con la línea del pelo tomada de su frente.



Figura 12: Los ocho cortes sobre tres retratos de dominio público.

8.2. Tinte y segmentación del pelo

La figura 13 compara las máscaras del YOLO propio y de MediaPipe, y el tinte con cada una:

- las dos máscaras coinciden en la mayor parte del pelo;
- el YOLO propio tiende a incluir más borde y más mechones finos.

Medida como IoU entre las máscaras sin refinar, la coincidencia fue de 0,82 a 0,94 en siete retratos de prueba. Dentro de la aplicación, que compara la máscara refinada de YOLO con la de

MediaPipe, el valor reportado varió entre 0,58 (pelo muy corto y rizado) y 0,96. El tinte conserva la textura del pelo y logra tanto aclarar como oscurecer.

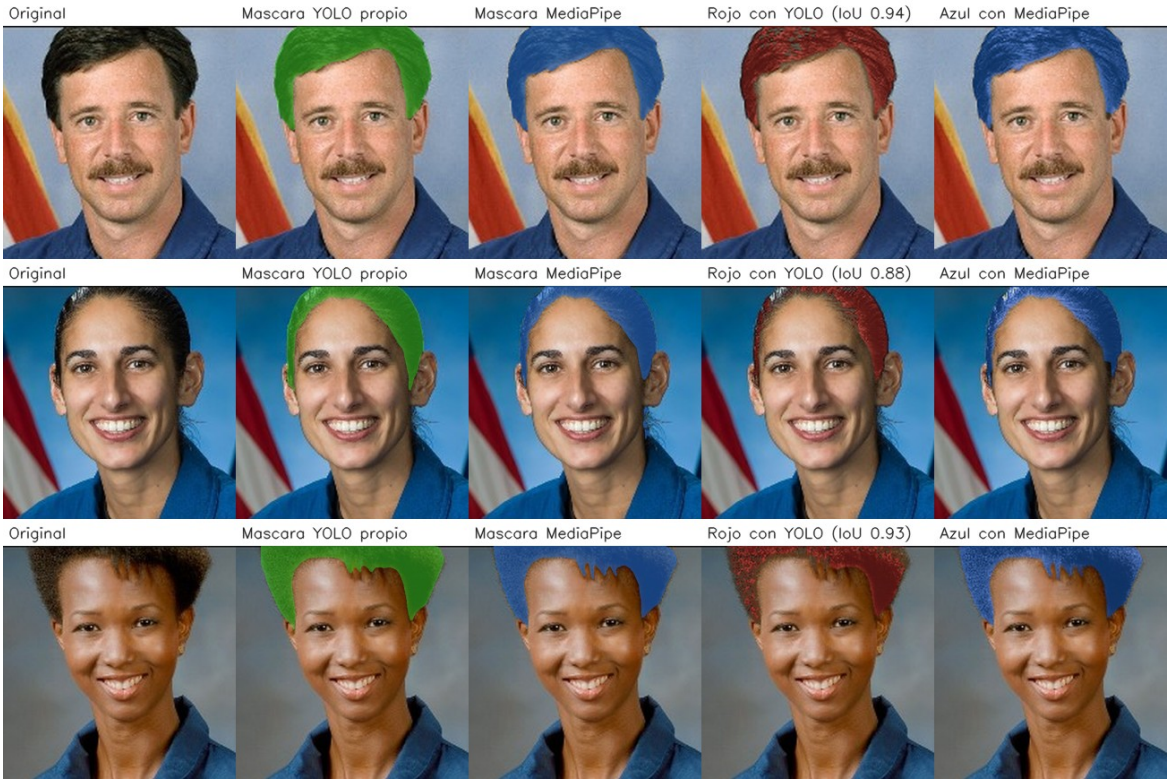


Figura 13: Máscara del YOLO propio (verde) y de MediaPipe (azul), y el tinte con cada una. El título de la cuarta columna indica la IoU entre ambas máscaras.

8.3. Tinte de la barba

La figura 14 muestra el tinte de barba en dos casos: una barba puesta por la propia aplicación, que se tiñe entera usando la máscara de su estilo, y un bigote real, detectado por textura.

8.4. Gaussianas 2D

La tabla 6 resume la calidad de la reconstrucción en seis retratos. La relación señal/ruido de pico (PSNR) crece casi linealmente con $\log N$: unos 8 dB por cada multiplicación por diez del número de gaussianas (figura 15).

Gaussianas	PSNR medio (dB)	Rango (dB)	Tiempo medio (s)
20	18,4	16,7 – 20,2	0,9
60	22,2	20,3 – 24,9	2,4
200	26,0	24,1 – 28,4	4,1
600	29,8	27,5 – 31,8	3,8
2000	34,5	32,9 – 35,9	3,6

Tabla 6: Reconstrucción de retratos con gaussianas 2D.



Figura 14: Tinte de barba. Arriba, un candado puesto por la aplicación; abajo, un bigote real.

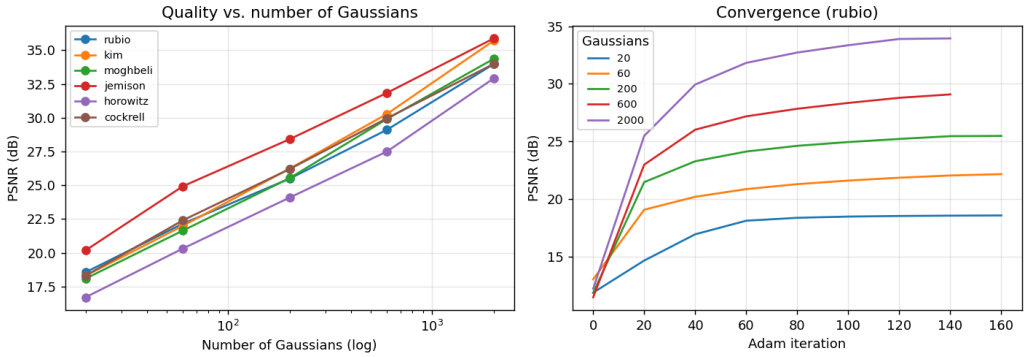


Figura 15: PSNR en función del número de gaussianas y convergencia por iteración.

8.5. Contrarreloj

Para validar el juego se programó un "bot barbero" que recorre en zigzag la zona que hay que afeitar y levanta la máquina en los saltos largos (tabla 7). Un bot que corta sin levantar la máquina, cruzando el objetivo, obtiene 0 puntos. Esto confirma que el puntaje premia afeitar con cuidado.

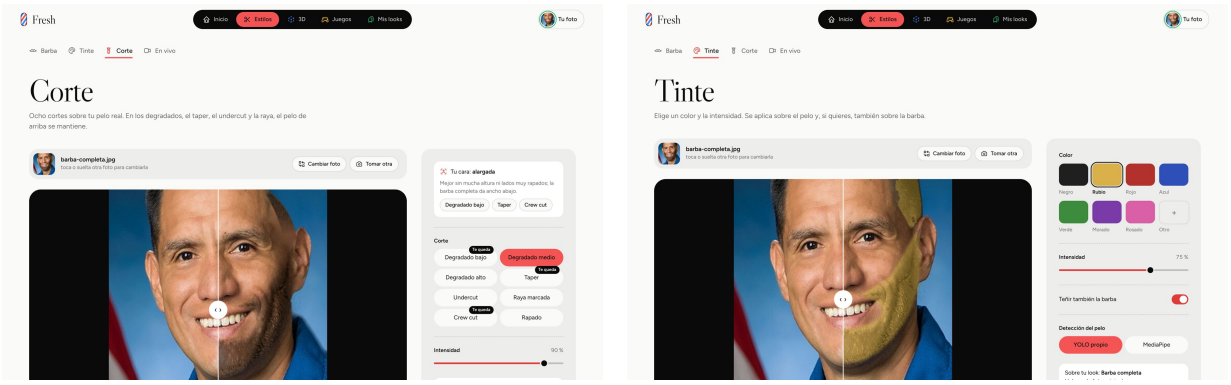
Retrato y objetivo	Puntaje	Limpieza	Conservación
rubio, candado	827	0,83	0,99
rubio, bigote	814	1,00	0,81
rubio, collar	690	0,69	1,00
kim, candado	817	0,83	0,99
horowitz, collar	656	0,66	1,00
cockrell, bigote	646	0,98	0,66

Tabla 7: Resultados del bot barbero.

8.6. Interfaz

La interfaz (figuras 1, 16 y 17) organiza las herramientas en cuatro grupos: *Estilos*, *3D*, *Juegos* y *Mis looks*. La foto es compartida entre todas las herramientas y cada resultado se ve en un

comparador antes/después. La tarjeta “Tu cara” muestra la forma detectada y marca los estilos sugeridos. La interfaz se adapta al celular.



(a) Corte, con sugerencias por forma de cara **(b)** Tinte con barba y selector de modelo

Figura 16: Paneles de Corte y Tinte sobre un look combinado.



Figura 17: Ficha para el barbero de un look combinado, con indicaciones de máquina por servicio.

8.7. Rendimiento

La tabla 8 reúne los tiempos medidos. Las fotos más grandes que 2048 px se reducen al recibirlas: una foto de 4000 px tardaba 18 s en el contenedor y ahora tarda menos de 1 s.

8.8. Pruebas automáticas

El proyecto tiene 47 pruebas con pytest, que GitHub Actions ejecuta en cada cambio:

- 30 de la API y de los módulos de estilo;
- 10 de *splatting* y física;

Operación	Tiempo
Corte, fotos de 1024 a 1638 px (contenedor Docker, CPU)	0,6 – 1,9 s
Corte, foto de 4000 px (contenedor Docker)	0,8 – 1,1 s
Corte, foto de 600 px (Apple M5, local)	0,1 – 0,3 s
Tinte con YOLO y barba (contenedor Docker, modelo ya cargado)	2,5 s
Construcción del espejo 3D (unas 105 000 gaussianas)	2,6 s
Barba en vivo, por cuadro	61 ms
Física de la barba en vivo, por cuadro	72 ms

Tabla 8: Tiempos medidos.

- 5 de la transferencia de pelo;
- 2 del laboratorio.

Entre otras cosas, verifican:

- **cortes**: ningún corte toca la cara; con intensidad 0 la foto no cambia; los píxeles lejos del pelo no cambian; un estilo desconocido da error;
- **entradas inválidas**: sin cara, la API responde 422, y con un estilo o modelo desconocido, 400;
- **forma de la cara**: la clasificación es correcta en proporciones sintéticas y todas las sugerencias corresponden a estilos existentes;
- **tinte de barba**: no tiñe piel limpia;
- **gaussianas 2D**: más gaussianas dan mayor PSNR;
- **física**: se asienta en reposo y se atrasa al mover la raíz;
- **espejo 3D**: las covarianzas son semidefinidas positivas y el `.ply` reconstruye exactamente la covarianza original;
- **juego**: no corta nada con la máquina apagada.

Además, la aplicación arranca y aprueba las pruebas tanto con YOLO instalado como sin él.

9. Limitaciones y trabajo futuro

Limitaciones conocidas.

- **Oclusiones.** Si una mano pasa delante de la cara en el modo en vivo, la barba se dibuja encima de la mano.
- **Pelo sobre los ojos.** En el rapado y el *crew cut*, si un flequillo largo tapa un ojo, quedan mechones sueltos, porque no se borra pelo dentro del polígono de la cara para no inventar el ojo.
- **Orejas tapadas.** Si el pelo tapaba las orejas, el rapado pone pelo corto donde irían las orejas, porque no hay información para reconstruirlas.
- **Barbas muy densas.** En barbas reales muy negras y tupidas, el tinte a veces toma solo los bordes: la textura no se distingue dentro de una zona uniforme.
- **Una sola vista en el espejo 3D.** El espejo se construye desde una foto: no tiene nuca, y con giros de más de unos 40° se estira el relieve detrás de la cara.

- **Velocidad en vivo.** El modo en vivo corre a unos 10 cuadros por segundo; con giros rápidos la barba se atrasa un poco.
- **Clasificación de la cara.** Es una heurística calibrada estadísticamente, no un diagnóstico; caras en el límite entre dos formas pueden caer en cualquiera de ellas.
- **Un solo usuario en vivo.** El estado del modo en vivo es único por servidor: dos cámaras a la vez compartirían el anclaje.

Trabajo futuro.

- Tinte en vivo en el navegador con un *shader* de WebGL y segmentación en JavaScript, para llegar a 30 cuadros por segundo sin servidor.
- Optimizar las gaussianas 3D con varias vistas (3DGS entrenado) para completar la cabeza.
- Segmentar la barba con un modelo propio, entrenado con las máscaras de CelebAMask-HQ.
- Estado del modo en vivo por sesión, para varios usuarios a la vez.

10. Conclusiones

Fresh muestra que un probador de barbería creíble se puede construir con técnicas clásicas de computación gráfica, sin modelos generativos y sin GPU. La clave fue modelar la geometría de la persona en lugar de pegar imágenes:

- una malla de Delaunay propia permite deformar barbas reales sobre cualquier cara;
- separar luminosidad y color en CIELAB permite teñir sin perder los mechones;
- un marco de la cabeza medido en anchos de cabeza, junto con un relleno *pull-push* que elige sus fuentes, permite quitar y volver a crear el pelo para ocho cortes distintos.

Las gaussianas resultaron ser una representación versátil a lo largo del proyecto: dibujan pelos sueltos en la barba con física, reconstruyen un retrato en 2D, forman una cabeza 3D girable y una peluca multivista, y siempre con el mismo principio de proyectar, acumular y componer.

Varias decisiones se tomaron midiendo:

- el anclaje de la textura a la malla redujo el deslizamiento de la barba en vivo, medido con flujo óptico [40], de 2,88 a 1,11;
- el cambio de Telea a *pull-push* eliminó el pelo fantasma;
- calibrar la forma de la cara con 419 caras evitó que todas cayeran en la misma clase;
- limitar el tamaño de entrada bajó de 18 s a menos de 1 s el tiempo de una foto de celular.

El producto está terminado y se puede reproducir:

- tiene interfaz para escritorio y celular;
- tiene 47 pruebas automáticas e integración continua;
- se levanta con un solo comando de Docker;
- incluye el modelo de segmentación entrenado por el equipo.

Las limitaciones que quedan (oclusiones, una sola vista en 3D, pelo que tapa los ojos o las orejas) marcan el camino para seguir mejorándolo.

Referencias

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. New York: Pearson, 4th ed., 2018.

[2] Commission Internationale de l'Éclairage, "Colorimetry," Tech. Rep. Publication 15:2004, CIE, Vienna, 2004.

[3] M. D. Fairchild, *Color Appearance Models*. Chichester: Wiley, 3rd ed., 2013.

[4] T. Porter and T. Duff, "Compositing digital images," in *Proc. SIGGRAPH '84, Computer Graphics*, vol. 18, pp. 253–259, 1984.

[5] S. Marschner and P. Shirley, *Fundamentals of Computer Graphics*. Boca Raton, FL: CRC Press, 4th ed., 2016.

[6] B. Delaunay, "Sur la sphère vide," *Bulletin de l'Académie des Sciences de l'URSS, Classe des Sciences Mathématiques et Naturelles*, no. 6, pp. 793–800, 1934.

[7] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*. Berlin: Springer, 3rd ed., 2008.

[8] S. Fortune, "A sweepline algorithm for Voronoi diagrams," *Algorithmica*, vol. 2, no. 1–4, pp. 153–174, 1987.

[9] A. Bowyer, "Computing Dirichlet tessellations," *The Computer Journal*, vol. 24, no. 2, pp. 162–166, 1981.

[10] D. F. Watson, "Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes," *The Computer Journal*, vol. 24, no. 2, pp. 167–172, 1981.

[11] R. Szeliski, *Computer Vision: Algorithms and Applications*. Cham: Springer, 2nd ed., 2022.

[12] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graphics*, vol. 42, no. 4, pp. 139:1–139:14, 2023.

[13] M. Zwicker, H. Pfister, J. van Baar, and M. Gross, "EWA volume splatting," in *Proc. IEEE Visualization*, pp. 29–36, 2001.

[14] X. Zhang, X. Ge, T. Xu, D. He, Y. Wang, H. Qin, G. Lu, J. Geng, and J. Zhang, "GaussianImage: 1000 FPS image representation and compression by 2D Gaussian splatting," in *Proc. European Conf. Computer Vision (ECCV)*, 2024.

[15] L. Verlet, "Computer "experiments" on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules," *Physical Review*, vol. 159, no. 1, pp. 98–103, 1967.

[16] T. Jakobsen, "Advanced character physics," in *Proc. Game Developers Conference*, (San José, CA), 2001.

[17] M. Müller, B. Heidelberger, M. Hennix, and J. Ratcliff, "Position based dynamics," *Journal of Visual Communication and Image Representation*, vol. 18, no. 2, pp. 109–118, 2007.

[18] A. Telea, "An image inpainting technique based on the fast marching method," *Journal of Graphics Tools*, vol. 9, no. 1, pp. 23–34, 2004.

[19] S. J. Gortler, R. Grzeszczuk, R. Szeliski, and M. F. Cohen, "The lumigraph," in *Proc. ACM SIGGRAPH*, pp. 43–54, 1996.

[20] J. Bigün and G. H. Granlund, "Optimal orientation detection of linear symmetry," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, pp. 433–438, 1987.

[21] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, vol. 1, pp. 511–518, 2001.

[22] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines." arXiv:1906.08172, 2019.

[23] Y. Kartynnik, A. Ablavatski, I. Grishchenko, and M. Grundmann, "Real-time facial surface geometry from monocular video on mobile GPUs." arXiv:1907.06724, 2019.

[24] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.

[25] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8." Software, versión 8, 2023. Disponible en: https://github.com/ultralytics/ultralytics.

[26] K. He, J. Sun, and X. Tang, "Guided image filtering," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 35, no. 6, pp. 1397–1409, 2013.

[27] S. Ramírez, "FastAPI." Software, 2018. Disponible en: https://fastapi.tiangolo.com.

[28] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

[29] C. R. Harris, K. J. Millman, S. J. van der Walt, *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[30] Khronos Group, "WebGL 2.0 specification." Especificación técnica, 2017. Disponible en: https://registry.khronos.org/webgl/specs/latest/2.0/.

[31] G. Casiez, N. Roussel, and D. Vogel, "1 € filter: A simple speed-based low-pass filter for noisy input in interactive systems," in *Proc. ACM SIGCHI Conf. Human Factors in Computing Systems (CHI)*, pp. 2527–2530, 2012.

[32] M. Svanera, U. R. Muhammad, R. Leonardi, and S. Benini, "Figaro, hair detection and segmentation in the wild," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, pp. 933–937, 2016.

[33] C.-H. Lee, Z. Liu, L. Wu, and P. Luo, "MaskGAN: Towards diverse and interactive facial image manipulation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 5549–5558, 2020.

[34] L. G. Farkas, *Anthropometry of the Head and Face.* New York: Raven Press, 2nd ed., 1994.

[35] P. Pérez, M. Gangnet, and A. Blake, "Poisson image editing," in *Proc. ACM SIGGRAPH*, pp. 313–318, 2003.

[36] G. M. Chaikin, "An algorithm for high-speed curve generation," *Computer Graphics and Image Processing*, vol. 3, no. 4, pp. 346–349, 1974.

[37] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[38] A. Laurentini, "The visual hull concept for silhouette-based image understanding," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 16, no. 2, pp. 150–162, 1994.

[39] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[40] G. Farnebäck, "Two-frame motion estimation based on polynomial expansion," in *Proc. Scandinavian Conf. Image Analysis (SCIA)*, pp. 363–370, 2003.

A. API del servidor

Todos los *endpoints* reciben `multipart/form-data` y viven bajo `/api`.

Endpoint	Uso
`GET /health`	Estado del servidor y si el YOLO propio está disponible.
`POST /face-shape`	Forma de la cara, proporciones y estilos sugeridos (JSON).
`POST /beard`	Barba sobre una foto (`style`, `strength`).
`POST /dye`	Tinte (`color`, `strength`, `beard`, `beard_style`, `model`). Devuelve en cabeceras el modelo usado y la IoU con MediaPipe.
`POST /haircut`	Corte (`style`, `strength`).
`POST /mesh`	Malla de Delaunay dibujada sobre la foto.
`POST /live`	Un cuadro del modo en vivo (tinte, barba, malla o física).
`POST /gaussians`	Retrato con N gaussianas 2D.
`POST /game/start`, `/game/frame`	Ronda de Contrarreloj.
`POST /head3d`	Gaussianas 3D del espejo, o archivo `.ply`.
`POST /transfer/*`	Escaneo, reconstrucción, cara 3D, fotos y peluca de gaussianas.
`POST /lab/build`, `/lab/mesh`	Avatar del laboratorio.
`GET /models/face_landmarker.task`	Modelo de malla facial para el navegador.

Tabla 9: *Endpoints* principales.

B. Estructura del código

```
backend/
  app/
    api/routes.py          endpoints
    core/pipeline.py       orquestacion
    core/detection/        cara, puntos, mano, filtro One-Euro
    core/segmentation/     pelo (MediaPipe, YOLO), barba, mascaras
    core/geometry/         Delaunay (Fortune, Bowyer-Watson), warp, forma de cara
    core/coloring/         tinte en CIELAB
    core/compositing/      barba, fisica, corte, cabeza sin pelo
    core/splatting/        gaussianas 2D, cabeza 3D, peluca 3D, cara 3D
    core/game/             Contrarreloj
    core/lab/              avatar en vivo
  models/                  modelos de MediaPipe y hair_best.pt
  tests/                   47 pruebas con pytest
  training/                preparacion de datos y entrenamiento de YOLO
frontend/
  index.html, css/, js/    interfaz y visor WebGL2
docs/                      documentacion tecnica y capturas
report/                    este documento en LaTeX
Dockerfile, docker-compose.yml, run.sh
```